Effect Systems & Interpreters

nterpreters · Read file → call service → write result; prod vs test interp · Build with

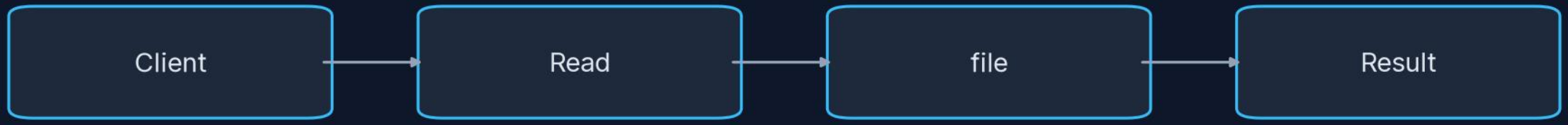
Today's Hook

- Today's force: Read file → call service → write result; prod vs test interpreters
- We earn Effect Interpreters by feeling the naive pain first.
- Demo main prints a real JDK 25 trace (happy + failure).
- Key types: EffectProgram, Eff, ReadFile, HttpGet

The Problem

- Program: read file → HTTP service → write result (AST of Eff).
- ProdInterpreter uses mutable FS + http function; TestInterpreter scripts responses.
- Naive pipeline hard-wires 'OK:'+content — cannot fail HTTP in tests.
- Success: /out gets SHIPPED:order-42; failHttp throws http-down.

Structure



Effect Interpreters: Read file → call service → write result; prod vs t

Real-World Scenario

- Running example: Read file → call service → write result; prod vs test interpreters
- Key types: EffectProgram, Eff, ReadFile, HttpGet
- Happy-path stdout cue: OK:order-42
- Failure-path stdout cue: --- test failure: http down ---

Naive Code

```java
package com.buildwithwaqar.patterns.functional.io.problem;

import java.util.Map;

/** Naive: effects inlined — cannot swap FS/HTTP for tests. */
public final class NaivePipeline {
    public static String run(Map<String, String> fs, String in, String out) {
        String content = fs.get(in);
        if (content == null) throw new IllegalStateException("missing");
        String resp = "OK:" + content; // hard-wired HTTP
        fs.put(out, resp);
        return resp;
    }
}
```

Why Naive Hurts

- Naive: effects inlined — cannot swap FS/HTTP for tests.
- Without Effect Interpreters, call sites absorb the wrong details.
- Failure mode to show on camera: --- test failure: http down ---
- New requirements force shotgun edits across the naive code.

Solution

```java
package com.buildwithwaqar.patterns.functional.io.problem;

import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.function.Function;

/**
 * Effect program: read a file → call a service → write a result.
 * Same AST runs under ProdInterpreter (in-memory FS+HTTP) and TestInterpreter (scripted).
 */
public final class EffectProgram {
    private EffectProgram() {}

    public sealed interface Eff permits ReadFile, HttpGet, WriteFile, Pure {}
    public record ReadFile(String path) implements Eff {}
    public record HttpGet(String url, String body) implements Eff {} // body = request payload from file
    public record WriteFile(String path, String content) implements Eff {}
    public record Pure(String value) implements Eff {}

    public abstract static class Prog<A> {
        private Prog() {}
        public abstract <B> Prog<B> flatMap(Function<? super A, Prog<B>> f);
        public final <B> Prog<B> map(Function<? super A, ? extends B> f) {
            return flatMap(a -> pure(f.apply(a)));
        }
```

Solution (2)

```
    public static <A> Prog<A> pure(A a) { return new Return<>(a); }
    public static Prog<String> readFile(String path) { return new Suspend<>(new ReadFile(path)); }
    public static Prog<String> httpGet(String url, String body) { return new Suspend<>(new HttpGet(url, body)); }
    public static Prog<Void> writeFile(String path, String content) { return new Suspend<>(new WriteFile(path, content)); }
}

private static final class Return<A> extends Prog<A> {
    final A value; Return(A value) { this.value = value; }
    public <B> Prog<B> flatMap(Function<? super A, Prog<B>> f) { return f.apply(value); }
}
private static final class Suspend<A> extends Prog<A> {
    final Eff eff; Suspend(Eff eff) { this.eff = eff; }
    public <B> Prog<B> flatMap(Function<? super A, Prog<B>> f) { return new Bind<>(this, f); }
}
private static final class Bind<A, B> extends Prog<B> {
    final Prog<A> prev; final Function<? super A, Prog<B>> f;
    Bind(Prog<A> prev, Function<? super A, Prog<B>> f) { this.prev = prev; this.f = f; }
    public <C> Prog<C> flatMap(Function<? super B, Prog<C>> g) {
        return new Bind<>(prev, a -> f.apply(a).flatMap(g));
    }
}
```

Solution (3)

```
    public static Prog<String> reportPipeline(String inPath, String url, String outPath) {
        return Prog.readFile(inPath).flatMap(content ->
                Prog
```

Runnable main

```java
package com.buildwithwaqar.patterns.functional.io.problem;

import java.util.HashMap;
import java.util.Map;

public final class ProblemDemo {
    public static void main(String[] args) {
        System.out.println("=== Effect Interpreters: read → HTTP → write ===");
        System.out.println("--- naive ---");
        var fs = new HashMap<String, String>();
        fs.put("/in.txt", "order-42");
        System.out.println(NaivePipeline.run(fs, "/in.txt", "/out.txt"));

        System.out.println("--- prod interpreter ---");
        var pfs = new HashMap<String, String>();
        pfs.put("/in.txt", "order-42");
        var prod = new EffectProgram.ProdInterpreter(pfs, body -> "SHIPPED:" + body);
        System.out.println(prod.run(EffectProgram.reportPipeline("/in.txt", "https://ship", "/out.txt")));
        System.out.println("log=" + prod.log());
        System.out.println("out=" + prod.fs().get("/out.txt"));

        System.out.println("--- test interpreter ---");
        var test = new EffectProgram.TestInterpreter(Map.of("/in.txt", "order-7"), "OK", false);
        System.out.println(test.run(EffectProgram.reportPipeline("/in.txt", "https://ship", "/out.txt")));
        System.out.println("writes=" + test.writes());
```

Runnable main (2)

```
        System.out.println("--- test failure: http down ---");
        try {
            new EffectProgram.TestInterpreter(Map.of("/in.txt", "x"), "OK", true)
                    .run(EffectProgram.reportPipeline("/in.txt", "https://ship", "/out.txt"));
        } catch (IllegalStateException e) {
            System.out.println("caught=" + e.getMessage());
        }
    }
}
```

Walkthrough Output

```
=== Effect Interpreters: read → HTTP → write ===
--- naive ---
OK:order-42
--- prod interpreter ---
SHIPPED:order-42
log=[read:/in.txt, http:https://ship, write:/out.txt]
out=SHIPPED:order-42
--- test interpreter ---
OK
writes={/out.txt=OK}
--- test failure: http down ---
caught=http-down
```

Step-by-Step Trace

- Match stdout to stages — look for: OK:order-42
- Then the injected failure: --- test failure: http down ---
- Re-run: java -cp ... 64-effect-interpreters ProblemDemo (see episode folder).
- Pause the video and predict the next printed line.

Modern Java 25 / Zazr

```java
package com.buildwithwaqar.patterns.functional.io.problem;

import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.function.Function;

/**
 * Effect program: read a file → call a service → write a result.
 * Same AST runs under ProdInterpreter (in-memory FS+HTTP) and TestInterpreter (scripted).
 */
public final class EffectProgram {
    private EffectProgram() {}

    public sealed interface Eff permits ReadFile, HttpGet, WriteFile, Pure {}
    public record ReadFile(String path) implements Eff {}
    public record HttpGet(String url, String body) implements Eff {} // body = request payload from file
    public record WriteFile(String path, String content) implements Eff {}
    public record Pure(String value) implements Eff {}

    public abstract static class Prog<A> {
        private Prog() {}
        public abstract <B> Prog<B> flatMap(Function<? super A, Prog<B>> f);
        public final <B> Prog<B> map(Function<? super A, ? extends B> f) {
            return flatMap(a -> pure(f.apply(a)));
        }
```

Modern Java 25 / Zazr (2)

```
    public static <A> Prog<A> pure(A a) { return new Return<>(a); }
    public static Prog<String> readF
```

Trade-offs

- Use Effect Interpreters when «Read file → call service → write result; prod vs test interpreters» repeats and a simpler feature does not erase it.
- Refuse extra types when a sealed switch or a function value already suffices.
- Teaching demos omit production metrics, retries, and multi-region clocks.
- Interview soundbite: state the force behind Effect Interpreters in product language.

Related Patterns

- Name the pattern usually confused with Effect Interpreters and separate the forces.
- Sketch the roles for «Read file → call service → write result; prod vs test interpreters» from memory in sixty seconds.
- Point to the next episode in this season on the playlist map.
- Link this episode's series/ folder in the YouTube description.

Recap

- Problem: Read file → call service → write result; prod vs test interpreters
- Naive pain → Effect Interpreters boundary → Demo stdout.
- Remember the failure cue: --- test failure: http down ---
- Break a ProblemTest assertion, watch it fail, restore it.